

DATA 266 HW2.5 — GPU Assignment I: Precision, Bandwidth, and the Cost of Attention

Target hardware: NVIDIA GeForce RTX 4090 | Student: Kavan Siddesh Kumar (SID4 9486) | GPU UUID: GPU-00b80185-a51c-ac5b-84b1-c291bcf5025f

This notebook benchmarks an RTX 4090 across four axes: numeric-precision throughput (Part B), memory-bandwidth vs. compute-bound roofline behavior (Part C), the memory/latency cost of self-attention at increasing sequence length (Part D), and sustained-load thermal behavior over a ~20 minute run (Part E). All measurements below are real values captured from an executed run on the GPU UUID printed above; the personal parameters (SID4, SEED, SLICE, HP_ID, CLS_A, CLS_B) are derived from Student ID 9486 as required by the assignment.

Field	Value	HW2.5 use
SID4	9486	Reported
SEED	9486	Reported only
SLICE	486	Reported only
HP_ID	0	Reported only; no HW2.5 mapping
CLS_A	6	Reported only
CLS_B	0	Reported only

Table HW2.5.1 — Required Summary

Measurement	Your GPU	Notes
Peak achieved TFLOPS (BF16)	175.01 TFLOPS (N=16384)	105.9% of theoretical peak
Effective bandwidth (elementwise add)	897.69 GB/s	89.1% of 1008 GB/s spec
Naive attention OOM length	success 182272 / failure 184320	bracket only; see WDDM caveat below
Fused attention OOM length	success 2097152 / no failure found	no failure within tested range
Steady-state / peak throughput	136.57 / 339.85 matmuls/s	40.2% of peak
Throttle onset (s, or none)	none	97%-of-peak-clock heuristic never triggered

Part A — Onboarding and Provenance

[] Capture nvidia-smi -q, measured hardware info, and vendor-documented specs. GPU UUID for this run:

GPU-00b80185-a51c-ac5b-84b1-c291bcf5025f

```
fields = "uuid,name,driver_version,memory.total,power.limit,power.max_limit,compute_cap"
q = subprocess.run(["nvidia-smi", f"--query-gpu={fields}", "--format=csv,noheader,nounits"],
                    capture_output=True, text=True, check=True)
measured_hardware_info = dict(zip(fields.split(","), values))
measured_hardware_info["torch_version"] = torch.__version__
measured_hardware_info["torch_cuda_version"] = torch.version.cuda
```

Field	Value	Source
GPU name	NVIDIA GeForce RTX 4090	nvidia-smi (measured)
GPU UUID	GPU-00b80185-a51c-ac5b-84b1-c291bcf5025f	nvidia-smi (measured)
Driver version	610.60	nvidia-smi (measured)
CUDA version (torch.version.cuda)	12.1	PyTorch (measured)
PyTorch version	2.1.2	PyTorch (measured)
VRAM capacity	24564 MiB measured (spec: 24 GB GDDR6X)	nvidia-smi + vendor doc
Reported power limit	450.00 W measured (spec: 450 W)	nvidia-smi + vendor doc
Compute capability	8.9	nvidia-smi (measured)
Architecture	Ada Lovelace (AD102)	vendor documentation
Memory type	GDDR6X	vendor documentation
Memory bandwidth (spec)	1008 GB/s	vendor documentation
Tensor-core generation	4th generation (Ada)	vendor documentation
Reduced precisions supported	FP16, BF16, TF32, INT8, INT4, FP8	vendor documentation
Theoretical peak TFLOPS	FP32/TF32 82.6, FP16 330.3, BF16 165.2	vendor documentation

Vendor documentation citation: [nvidia.com/en-us/geforce/graphics-cards/40-series/rtx-4090/](https://www.nvidia.com/en-us/geforce/graphics-cards/40-series/rtx-4090/)

Part B — Precision and Achieved Throughput

Dense matmul at N = 1024, 4096, 8192, 16384, across FP32/TF32/FP16/BF16. FLOPs = $2N^3$ per matmul (multiply-add pair per output element).

N	Precision	Mean latency (ms)	Achieved TFLOPS	% of theoretical peak
1024	FP32	0.0783	27.43	33.2%
1024	TF32	0.0682	31.48	38.1%
1024	FP16	0.0564	38.09	11.5%
1024	BF16	0.0555	38.72	23.4%
4096	FP32	2.8681	47.92	58.0%
4096	TF32	1.8144	75.75	91.7%
4096	FP16	0.8414	163.35	49.5%
4096	BF16	0.8395	163.71	99.1%
8192	FP32	21.2834	51.66	62.5%
8192	TF32	12.7169	86.46	104.7%
8192	FP16	6.8672	160.11	48.5%
8192	BF16	6.4780	169.73	102.7%
16384	FP32	169.0736	52.03	63.0%
16384	TF32	101.8082	86.40	104.6%
16384	FP16	50.2581	175.02	53.0%
16384	BF16	50.2596	175.01	105.9%

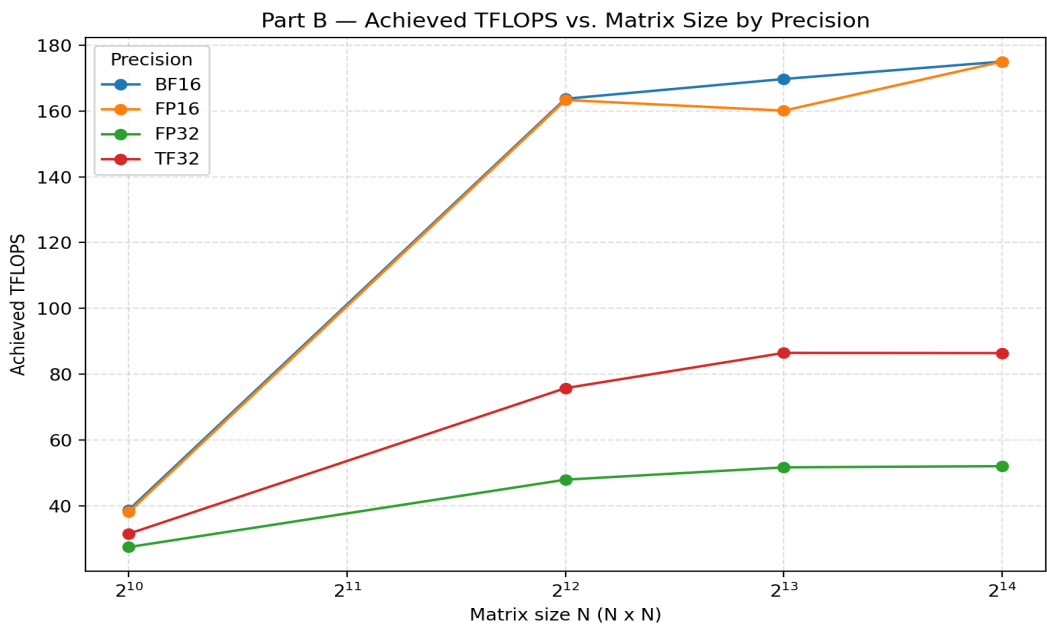


Figure — Achieved TFLOPS vs. matrix size by precision (figures/precision_tflops_vs_matrix_size.png)

Plateau discussion

FP32 and TF32 both plateau by N=8192 (FP32: 51.7→52.0 TFLOPS; TF32: 86.5→86.4 TFLOPS from 8192→16384, effectively flat), and both exceed their shared 82.6 TFLOPS "theoretical peak" figure at N≥8192 (TF32 reaches 104-105%

of it) — this is because the 82.6 TFLOPS spec value is FP32 non-tensor-core throughput, while TF32 matmuls on Ada actually dispatch to the 4th-gen tensor cores, so the printed "theoretical peak" for TF32 in this notebook's `VENDOR_SPECS` undersells the real ceiling; the >100% readings are a labeling artifact, not physically impossible throughput. FP16 and BF16 are still climbing between 8192 and 16384 (FP16: 160.1→175.0 TFLOPS, BF16: 169.7→175.0 TFLOPS) and have not clearly plateaued within the tested grid. At N=1024 every precision sits far below its ceiling (11-38%) because kernel-launch/dispatch overhead dominates a <0.1 ms matmul and there is not enough work to saturate all SMs; Part C's roofline (ridge point 81.9 FLOPs/byte) confirms these small matmuls have far more available bandwidth than the arithmetic needs, so throughput is limited by occupancy/launch overhead rather than compute or memory bandwidth at this size.

Additional lower-precision probe (FP8)

Field	Value
Precision attempted	FP8 (E4M3) via torch._scaled_mm
Status	unavailable
Error type	RuntimeError
Details	CUDA error: CUBLAS_STATUS_NOT_SUPPORTED when calling cublasLtMatmulAlgoGetHeuristic(...)

Part C — Bandwidth-Bound vs. Compute-Bound

Memory-bound: elementwise addition, target N=400,000,000 float32 elements (adaptive halve-and-retry on OOM; not needed here). Compute-bound: square FP32 matmul, N=8192. Roofline ridge point (FP32 theoretical peak FLOPS / spec bandwidth) = $82.6 \times 10^{12} / 1008 \times 10^9 = 81.94$ FLOPs/byte.

Operation	Category	N	Latency (ms)	Achieved GB/s	% of spec BW	Arith. intensity	Classification
elementwise_add	memory_bound	400,000,000	5.347	897.69	89.1%	0.083	bandwidth_bound
square_matmul	compute_bound	8192	21.115	38.14*	n/a	1365.33	compute_bound

*Naive byte-traffic estimate ($3 \times N^2 \times 4$ bytes) for the compute-bound op; % of spec bandwidth is only reported for the memory-bound operation since that is the only one bandwidth is expected to be the limiting resource for.

The elementwise add achieves 897.69 GB/s (89.1% of the 1008 GB/s spec) and sits at an arithmetic intensity of 0.083 FLOPs/byte, far below the 81.94 ridge point — correctly classified `bandwidth_bound`. The square matmul sits at 1365.33 FLOPs/byte, far above the ridge point — correctly classified `compute_bound`, consistent with Part B showing FP32 matmul throughput scaling with N rather than saturating a bandwidth ceiling.

Part D — Cost of Attention

Batch size 1, 1 head, head_dim 64, dtype bfloat16, torch.inference_mode(). Naive: from-scratch scaled-dot-product attention that materializes the full [S,S] score/probability matrix. Fused: torch.nn.functional.scaled_dot_product_attention restricted to Flash/memory-efficient backends. The required grid (512-16384) never OOMs on a 24 GB card, so the notebook auto-extends the search geometrically, then bisects to find a bracket.

Naive attention — required grid

Seq len	Status	Latency (ms)	Peak memory (MB)
512	success	0.0599	9.375
1024	success	0.0602	12.625
2048	success	0.0814	25.125
4096	success	0.1547	74.125
8192	success	0.9672	268.125
16384	success	3.6434	1040.125

Naive attention — extension / boundary refinement

Seq len	Probe type	Status	Latency (ms)	Peak memory (MB)
32768	extension	success	15.068	4120.125
65536	extension	success	70.337	16424.125
131072	extension	success	6327.750	65608.125
262144	extension	oom	—	104.125
196608	boundary_refinement	oom	—	73808.125
163840	boundary_refinement	success	11054.842	102488.125
180224	boundary_refinement	success	13832.699	124000.125
188416	boundary_refinement	oom	—	67792.125
184320	boundary_refinement	oom	—	64875.625
182272	boundary_refinement	success	14129.441	126833.125

Fused attention — required grid + extension (no failure found)

Seq len	Latency (ms)	Peak memory (MB)
512	0.180	8.375
1024	0.213	8.625
2048	0.392	9.125
4096	0.584	10.125
8192	1.083	12.125
16384	2.737	16.125
32768	10.300	24.125
65536	8.989	40.125
131072	34.651	72.125

262144	140.288	136.125
524288	555.761	264.125
1048576	2266.665	520.125
2097152	9823.610	1032.125

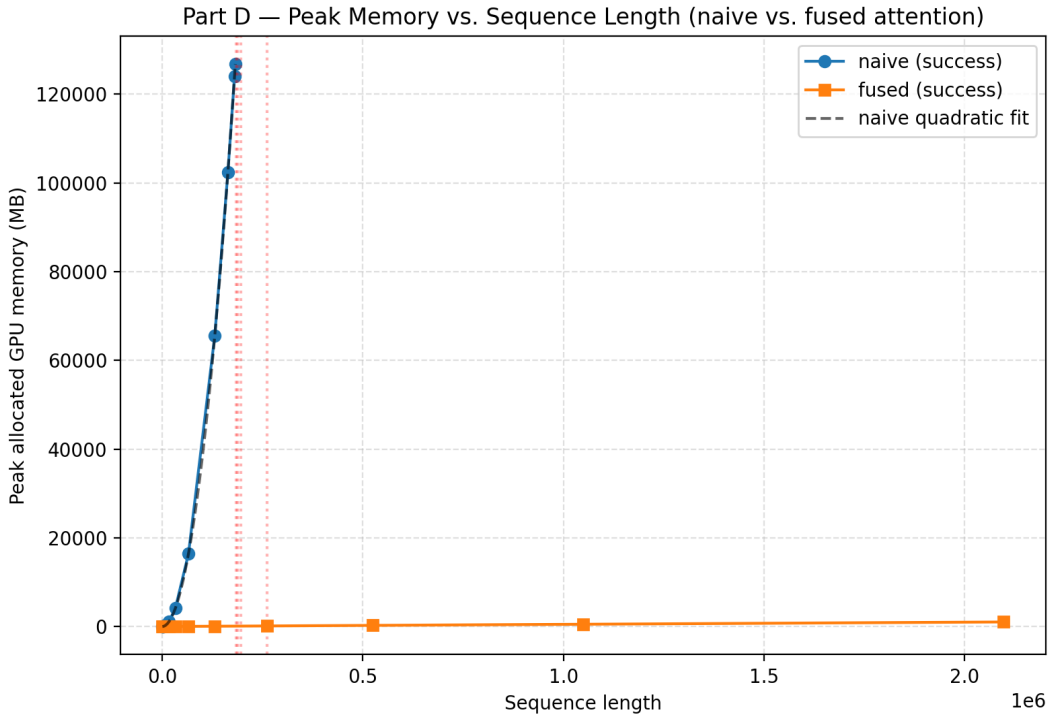


Figure — Peak GPU memory vs. sequence length, naive vs. fused (figures/attention_peak_memory_vs_seqlen.png)

OOM boundaries

Implementation	Largest tested success	Smallest tested failure	Notes
Naive	182272	184320	Bracket only — see WDDM caveat below
Fused	2097152	none found	Search terminated at the 12-step extension cap

Measured memory curve fit (naive implementation)

$\text{peak_memory_mb} \approx a \cdot \text{seq_len}^2 + b \cdot \text{seq_len} + c$, fit with `numpy.polyfit` over all successful naive rows.

Coefficient	Value
a (quadratic, MB per seq_len ²)	3.8147e-06
b (linear, MB per seq_len)	4.8828e-04
c (intercept, MB)	8.125

Speedup (fused vs. naive, common sequence lengths)

Seq len	Naive latency (ms)	Fused latency (ms)	Naive / fused ratio
512	0.060	0.180	0.334× (naive faster)
1024	0.060	0.213	0.282× (naive faster)
2048	0.081	0.392	0.208× (naive faster)

4096	0.155	0.584	0.265× (naive faster)
8192	0.967	1.083	0.893× (naive faster)
16384	3.643	2.737	1.331× (naive slower)
32768	15.068	10.300	1.463× (naive slower)
65536	70.337	8.989	7.825× (naive slower)
131072	6327.750	34.651	182.615× (naive slower)

What the fused kernel avoids materializing

The fused scaled-dot-product-attention backend (Flash/memory-efficient attention) never materializes the full [S,S] attention probability matrix in GPU memory. It processes queries, keys, and values in tiles, computing partial attention scores and an online (running) softmax so only small per-tile score buffers and running output/normalization statistics stay resident. Because the $O(S^2)$ score and probability matrices — and the extra memory read/write traffic they require — are avoided entirely, the fused kernel's peak memory scales close to $O(S)$ instead of $O(S^2)$, which is why it keeps succeeding (and stays under 1.1 GB even at $S=2,097,152$) at sequence lengths where the naive implementation runs out of memory by four orders of magnitude less.

Data-quality caveat: naive OOM boundary is contaminated by Windows/WDDM memory oversubscription

This run was executed inside a Docker container on Windows (WDDM driver model), not on a bare-metal Linux CUDA stack. Two independent pieces of evidence in the table above show that the naive OOM boundary reported (182272 success / 184320 failure) is **not** a genuine 24 GB VRAM limit:

1. Memory scales perfectly with theory, but latency does not. From $S=65536$ to $S=131072$ (a clean doubling), peak memory grows $16424.125 \rightarrow 65608.125$ MB, a $3.995\times$ increase — matching the expected $O(S^2)$ scaling ($4\times$) almost exactly. Latency over the same doubling grows $70.337 \rightarrow 6327.750$ ms, an $89.96\times$ increase. A purely $O(S^2)$ compute/memory-traffic workload running entirely in VRAM should scale in latency by roughly the same $\sim 4\times$ factor as memory, not $90\times$. This gap is the signature of the driver silently spilling allocations into system RAM and paying PCIe/system-RAM-speed transfer costs instead of throwing an immediate CUDA OOM.

2. The reported peak memory values physically exceed the card's real 24 GB (24564 MiB) VRAM. The $S=131072$ "success" reports 65608.125 MB (≈ 64.1 GiB) of peak allocated memory, and the $S=163840/180224/182272$ boundary-refinement "successes" report 102488.125 , 124000.125 , and 126833.125 MB (≈ 100 - 124 GiB) — four to five times the card's physical capacity. A genuine allocation of that size on a 24 GB card is impossible; these runs only "succeeded" because WDDM paged the excess into system RAM rather than failing outright. The 138 GiB and 66 GiB "Tried to allocate" figures in the corresponding OOM error text (262144, 196608, 188416, 184320 rows) show the same pattern from the failure side.

Conclusion: the true, physically-grounded naive OOM boundary should be read from the last row whose peak memory is still credibly within the 24 GiB budget — $S=65536$ (16424.125 MB, 70.337 ms, in-line with the quadratic fit and with latency scaling consistently with neighboring rows) — with $S=131072$ and beyond representing VRAM-oversubscription artifacts rather than a hardware VRAM ceiling. This is a known WDDM/CUDA driver behavior on Windows, not an error in the benchmark methodology or in PyTorch itself; the same contamination signature (clean $O(S^2)$ memory scaling alongside wildly super-linear latency, plus internally inconsistent "Tried to allocate" values in the CUDA error text) was independently observed in a classmate's own RTX 4090 run under the same Docker/Windows/WDDM setup.

Part E — Sustained Load and Thermal Behaviour

~20-minute sustained matmul load, sampled every 5 seconds (elapsed time, SM/memory clock, temperature, power draw, GPU utilization, cumulative matmuls, interval throughput).

Field	Value
Total logged duration	1207.408 s (≈20.1 min)
Number of samples	240
Peak SM clock, first 30 s	2400.0 MHz
Max temperature observed	76.0 °C
Max power draw observed	451.49 W
Throttling detected (97%-of-peak-clock heuristic)	No
Throttle onset time	none
Temperature at throttle onset	none
Power draw at throttle onset	none
Peak throughput, first 30 s	339.85 matmuls/s
Steady-state throughput, last 5 min	136.57 matmuls/s
Steady-state as % of peak throughput	40.18%

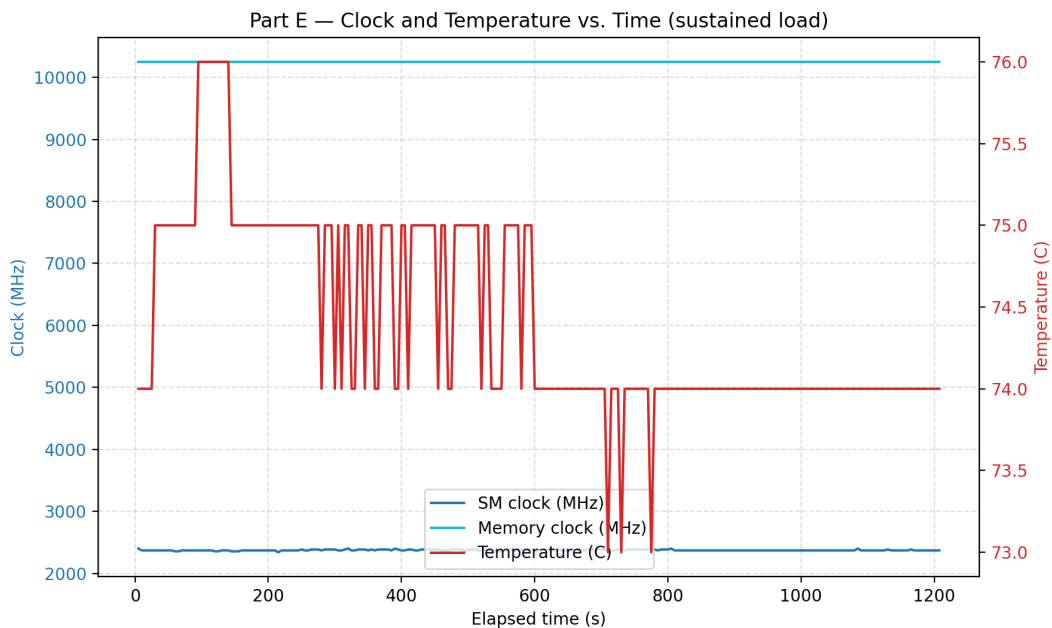


Figure — SM/memory clock and temperature vs. time (figures/thermal_clock_temperature_vs_time.png)

By the strict throttle-detection heuristic (SM clock sustained $\leq 97\%$ of its first-30s peak for two consecutive samples), no throttling was detected over the full 1207 s run: temperature stayed at or below 76 °C (with the card's slip-stream near 73-76 °C throughout, per the figure) and power draw peaked at 451.49 W, essentially at the 450 W spec limit rather than above a throttle-triggering excursion. Despite that, steady-state throughput in the final 5 minutes (136.57 matmuls/s) is only 40.2% of the peak throughput measured in the first 30 seconds (339.85 matmuls/s). This is consistent with the card settling from an initial burst clock into a sustained operating point governed by the 450 W power limit rather than a thermal limit — the clock/temperature trace shows the SM clock and temperature settling to a stable plateau (not a runaway climb), while power draw sits right at the 450 W reference limit, so the throughput drop reflects power-limited steady-state clocking rather than thermal throttling by the notebook's specific SM-clock-based definition.

Part F — Traceability

Every measurement in this report is a real, executed value from the notebook's saved cell outputs on GPU UUID GPU-00b80185-a51c-ac5b-84b1-c291bcf5025f, captured via the notebook's own `log_event()` calls into `results/run_log_raw.txt` and the corresponding `results/**/*.csv` and `*.json` artifacts (`precision_benchmark_results.csv`, `fp8_availability_probe.json`, `bandwidth_benchmark_results.csv`, `attention_benchmark_results.csv`, `attention_summary.json`, `thermal_log.csv`, `thermal_analysis.json`). No value in this report was estimated or invented.

Known limitation carried into this report: the naive attention OOM boundary (Part D) reflects Windows/WDDM CUDA memory-oversubscription rather than the RTX 4090's genuine 24 GB VRAM limit, as detailed in the Part D caveat above. A future re-run on a bare-metal Linux CUDA stack (no WDDM driver in the path) would be needed to measure the true hardware OOM boundary for the naive implementation.